# Module 2 – JavaScript Fundamentals

## 1. Introduction to JavaScript

JavaScript is a high-level, interpreted programming language used to make web pages interactive and dynamic.

It is mainly used along with HTML and CSS to develop interactive web applications.

**Uses of JavaScript:**

- Handling button clicks and user events.
- Validating forms.
- Updating HTML content dynamically.
- Manipulating HTML and CSS.
- Communicating with servers using APIs.
- Developing frontend and backend applications.

**Example:**

```
None
<!DOCTYPE html>
<html>
<head>
    <title>JavaScript Example</title>
</head>
<body>
    <h2 id="heading">Hello World</h2>

    <button onclick="changeText()">Click Me</button>

    <script>
        function changeText() {
            document.getElementById("heading").innerHTML = "Welcome
to JavaScript!";
        }
    </script>
</body>
</html>
```

## 2. Adding JavaScript to HTML

JavaScript can be added to HTML in three ways.

## 2.1 Inline JavaScript

JavaScript is written directly inside an HTML element using event attributes.

```
None
<button onclick="alert('Hello World')">Click Me</button>
```

## 2.2 Internal JavaScript

JavaScript is written inside the `<script>` tag in the HTML document.

```
None
<!DOCTYPE html>
<html>
<body>
    <h2>JavaScript Example</h2>

    <script>
        console.log("Hello JavaScript");
    </script>
</body>
</html>
```

## 2.3 External JavaScript

JavaScript is written in a separate `.js` file and connected to HTML using the `src` attribute.

**index.html**

```
None
<script src="script.js"></script>
```

**script.js**

```
None
console.log("External JavaScript File");
```

**Advantages:**

- Improves code reusability.
- Separates HTML and JavaScript.
- Makes code easier to maintain.

# 3. JavaScript Syntax

JavaScript syntax refers to the rules used to write JavaScript programs.

**Example:**

```
None
let name = "Divya";
let age = 21;

console.log(name);
console.log(age);
```

**Important syntax rules:**

- JavaScript is case-sensitive.
- Statements can end with a semicolon.
- Variables are declared using `var`, `let`, or `const`.
- Code blocks are enclosed within curly braces `{}`.
- Comments are used to explain code.

**Comments:**

```
None
// Single-line comment

/*
Multi-line comment
*/
```

# 4. Variables in JavaScript

Variables are used to store data values in a program.

JavaScript provides three keywords to declare variables.

## 4.1 var

- Function-scoped.

- Can be redeclared.
- Can be reassigned.

```
None
var name = "Divya";
var name = "Priya";

console.log(name);
```

### 4.2 let

- Block-scoped.
- Cannot be redeclared within the same scope.
- Can be reassigned.

```
None
let age = 21;
age = 22;

console.log(age);
```

### 4.3 const

- Block-scoped.
- Cannot be redeclared within the same scope.
- Cannot be reassigned.

```
None
const country = "India";

console.log(country);
```

**Difference between var, let, and const**

| Feature | var | let | const |
|---|---|---|---|
| Scope | Function | Block | Block |
| Redeclaration | Yes | No | No |
| Reassignment | Yes | Yes | No |

| | | | |
|---|---|---|---|
| Initialization required | No | No | Yes |

# 5. JavaScript Data Types

Data types define the type of value stored in a variable.

JavaScript has two main categories of data types.

## 5.1 Primitive Data Types

Primitive data types store single values.

**1. String**

Used to store text.

```
None
let name = "Divya";
```

**2. Number**

Used to store integer and decimal values.

```
None
let age = 21;
let price = 99.50;
```

**3. Boolean**

Stores either `true` or `false`.

```
None
let isActive = true;
```

**4. Undefined**

A variable declared without assigning a value has the value `undefined`.

```
None
let city;
console.log(city);
```

**5. Null**

Represents an intentional absence of a value.

```
None
let selectedValue = null;
```

**6. BigInt**

Used to represent very large integer values.

```
None
let number = 12345678901234567890n;
```

**7. Symbol**

Used to create unique identifiers.

```
None
let id = Symbol("id");
```

## 5.2 Non-Primitive Data Types

**1. Object**

Stores data in key-value pairs.

```
None
let student = {
    name: "Divya",
    age: 21,
    department: "CSE"
};
```

**2. Array**

Stores multiple values in a single variable.

```
None
let subjects = ["Java", "C#", "JavaScript"];
```

### 3. Function

A function is a reusable block of code.

```
None
function greet() {
    console.log("Welcome");
}
```

# 6. JavaScript Operators

Operators are symbols used to perform operations on variables and values.

## 6.1 Arithmetic Operators

Used to perform mathematical operations.

| Operator | Meaning | Example |
|---|---|---|
| + | Addition | 10 + 5 |
| - | Subtraction | 10 - 5 |
| * | Multiplication | 10 * 5 |
| / | Division | 10 / 5 |
| % | Modulus | 10 % 3 |
| ** | Exponentiation | 2 ** 3 |
| ++ | Increment | x++ |
| -- | Decrement | x-- |

## 6.2 Assignment Operators

Used to assign values to variables.

```
None
let x = 10;

x += 5;  // x = x + 5
x -= 2;  // x = x - 2
```

```
x *= 2;  // x = x * 2
x /= 2;  // x = x / 2
```

### 6.3 Comparison Operators

Used to compare two values.

| Operator | Meaning |
|---|---|
| == | Equal value |
| === | Equal value and type |
| != | Not equal value |
| !== | Not equal value or type |
| > | Greater than |
| < | Less than |
| >= | Greater than or equal |
| <= | Less than or equal |

**Example:**

None
```
let a = 10;
let b = "10";

console.log(a == b);  // true
console.log(a === b); // false
```

### 6.4 Logical Operators

Used to combine multiple conditions.

| Operator | Meaning |
|---|---|
| && | Logical AND |
| \|\| | Logical OR |

| | |
|---|---|
| ! | Logical NOT |

**Example:**

```
None
let age = 21;

console.log(age >= 18 && age <= 60);
```

### 6.5 Ternary Operator

The ternary operator is a shorthand for writing simple conditional statements.

**Syntax:**

```
None
condition ? expression1 : expression2;
```

**Example:**

```
None
let age = 20;

let result = age >= 18 ? "Eligible" : "Not Eligible";

console.log(result);
```

## 7. Conditional Statements

Conditional statements execute different blocks of code based on conditions.

### 7.1 if Statement

```
None
let age = 20;

if (age >= 18) {
    console.log("Eligible to vote");
}
```

### 7.2 if-else Statement

```
None
let age = 16;

if (age >= 18) {
    console.log("Eligible");
} else {
    console.log("Not Eligible");
}
```

### 7.3 else-if Statement

```
None
let mark = 85;

if (mark >= 90) {
    console.log("Grade A");
} else if (mark >= 75) {
    console.log("Grade B");
} else if (mark >= 50) {
    console.log("Grade C");
} else {
    console.log("Fail");
}
```

### 7.4 switch Statement

Used to execute one block from multiple possible cases.

```
None
let day = 2;

switch (day) {
    case 1:
        console.log("Monday");
        break;

    case 2:
        console.log("Tuesday");
```

```
        break;

    default:
        console.log("Invalid Day");
}
```

# 8. Loops in JavaScript

Loops are used to execute a block of code repeatedly.

## 8.1 for Loop

Used when the number of iterations is known.

```
None
for (let i = 1; i <= 5; i++) {
    console.log(i);
}
```

## 8.2 while Loop

Executes as long as the condition is true.

```
None
let i = 1;

while (i <= 5) {
    console.log(i);
    i++;
}
```

## 8.3 do-while Loop

Executes the block at least once before checking the condition.

```
None
let i = 1;
```

```
do {
    console.log(i);
    i++;
} while (i <= 5);
```

### 8.4 for...of Loop

Used to iterate over iterable values such as arrays and strings.

```
None
let subjects = ["Java", "C#", "Python"];

for (let subject of subjects) {
    console.log(subject);
}
```

### 8.5 for...in Loop

Used to iterate over the enumerable property keys of an object.

```
None
let student = {
    name: "Divya",
    age: 21
};

for (let key in student) {
    console.log(key, student[key]);
}
```

### 8.6 break and continue

**break:** Terminates the loop immediately.

```
None
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        break;
```

```
    }

    console.log(i);
}
```

**continue:** Skips the current iteration and moves to the next iteration.

```
None
for (let i = 1; i <= 5; i++) {
    if (i === 3) {
        continue;
    }

    console.log(i);
}
```

# 9. Functions in JavaScript

A function is a reusable block of code designed to perform a particular task.

## 9.1 Function Declaration

```
None
function greet() {
    console.log("Hello");
}

greet();
```

## 9.2 Function with Parameters

Parameters are variables used to receive values passed to a function.

```
None
function add(a, b) {
    console.log(a + b);
}
```

```
add(10, 20);
```

### 9.3 Function with Return Value

The `return` statement sends a value back to the calling code.

```
None
function add(a, b) {
    return a + b;
}

let result = add(10, 20);

console.log(result);
```

### 9.4 Function Expression

A function can be assigned to a variable.

```
None
const greet = function() {
    console.log("Welcome");
};

greet();
```

### 9.5 Arrow Function

Arrow functions provide a shorter syntax for writing functions.

```
None
const add = (a, b) => {
    return a + b;
};

console.log(add(10, 20));
```

Shorter syntax:

```
None
const square = n => n * n;

console.log(square(5));
```

# 10. Scope in JavaScript

Scope determines where a variable can be accessed in a program.

## 10.1 Global Scope

A variable declared outside all functions and blocks can generally be accessed throughout its accessible script or module scope.

```
None
let name = "Divya";

function display() {
    console.log(name);
}
```

## 10.2 Function Scope

Variables declared with `var` inside a function are accessible within that function.

```
None
function display() {
    var age = 21;
    console.log(age);
}
```

## 10.3 Block Scope

Variables declared using `let` and `const` inside a block are accessible only within that block.

```
None
if (true) {
    let age = 21;
    const name = "Divya";
```

```
}
```

# 11. Arrays in JavaScript

An array is a collection used to store multiple values in a single variable.

## 11.1 Creating an Array

```
None
let fruits = ["Apple", "Banana", "Orange"];
```

## 11.2 Accessing Array Elements

Array indexing starts from zero.

```
None
console.log(fruits[0]); // Apple
console.log(fruits[1]); // Banana
```

## 11.3 Array Properties and Methods

**length:** Returns the number of elements.

```
None
console.log(fruits.length);
```

**push():** Adds an element at the end.

```
None
fruits.push("Mango");
```

**pop():** Removes the last element.

```
None
fruits.pop();
```

**unshift():** Adds an element at the beginning.

```
None
fruits.unshift("Grapes");
```

**shift():** Removes the first element.

```
None
fruits.shift();
```

**includes():** Checks whether an element exists.

```
None
console.log(fruits.includes("Apple"));
```

**indexOf():** Returns the index of an element or -1 if it is not found.

```
None
console.log(fruits.indexOf("Banana"));
```

**slice():** Returns a portion of an array without modifying the original array.

```
None
let selected = fruits.slice(0, 2);
```

**splice():** Adds, removes, or replaces elements in the original array.

```
None
fruits.splice(1, 1);
```

# 12. Array Iteration Methods

## 12.1 forEach()

Executes a function for each array element. It does not create a new array.

```
None
let numbers = [1, 2, 3, 4];
```

```
numbers.forEach(num => {
    console.log(num);
});
```

### 12.2 map()

Creates a new array by transforming each element.

```
None
let numbers = [1, 2, 3];

let result = numbers.map(num => num * 2);

console.log(result);
```

### 12.3 filter()

Creates a new array containing elements that satisfy a condition.

```
None
let numbers = [10, 15, 20, 25];

let result = numbers.filter(num => num > 15);

console.log(result);
```

### 12.4 find()

Returns the first element that satisfies a condition.

```
None
let numbers = [10, 20, 30, 40];

let result = numbers.find(num => num > 20);

console.log(result);
```

### 12.5 reduce()

Reduces an array into a single value by applying a function.

```
None
let numbers = [10, 20, 30];

let total = numbers.reduce((sum, num) => sum + num, 0);

console.log(total);
```

# 13. Objects in JavaScript

An object is a collection of properties represented as key-value pairs.

## 13.1 Creating an Object

```
None
let student = {
    id: 101,
    name: "Divya",
    department: "CSE",
    age: 21
};
```

## 13.2 Accessing Object Properties

**Dot notation:**

```
None
console.log(student.name);
```

**Bracket notation:**

```
None
console.log(student["department"]);
```

## 13.3 Adding Properties

```
None
student.email = "divya@gmail.com";
```

### 13.4 Updating Properties

```
None
student.age = 22;
```

### 13.5 Deleting Properties

```
None
delete student.age;
```

### 13.6 Object Methods

A function defined inside an object is called an object method.

```
None
let student = {
    name: "Divya",

    display: function() {
        console.log(this.name);
    }
};

student.display();
```

## 14. Destructuring

Destructuring is a feature used to extract values from arrays or properties from objects into separate variables.

### 14.1 Array Destructuring

```
None
let colors = ["Red", "Green", "Blue"];

let [first, second, third] = colors;

console.log(first);
console.log(second);
```

### 14.2 Object Destructuring

```
None
let student = {
    name: "Divya",
    age: 21
};

let { name, age } = student;

console.log(name);
console.log(age);
```

## 15. Spread and Rest Operators

Both spread and rest use three dots (`...`), but they have different purposes.

### 15.1 Spread Operator

The spread operator expands elements of an array or properties of an object.

**Array Example:**

```
None
let a = [1, 2, 3];
let b = [...a, 4, 5];

console.log(b);
```

**Object Example:**

```
None
let student = {
    name: "Divya",
    age: 21
};

let updatedStudent = {
    ...student,
    department: "CSE"
```

```
};
```

### 15.2 Rest Operator

The rest operator collects multiple values into a single array or object.

```
None
function add(...numbers) {
    return numbers.reduce((sum, num) => sum + num, 0);
}

console.log(add(10, 20, 30));
```

## 16. Template Literals

Template literals allow variables and expressions to be embedded inside strings using backticks.

**Syntax:** `${expression}`

```
None
let name = "Divya";
let age = 21;

console.log(`My name is ${name} and my age is ${age}`);
```

**Advantages:**

- Supports variable interpolation.
- Supports multiline strings.
- Improves string readability.

## 17. String Methods

String methods are used to perform operations on text values.

```
None
let text = "JavaScript Fundamentals";
```

| Method | Purpose |
| --- | --- |
| toUpperCase() | Converts to uppercase |
| toLowerCase() | Converts to lowercase |
| trim() | Removes whitespace from both ends |
| includes() | Checks whether text exists |
| indexOf() | Returns the position of text |
| slice() | Extracts a portion of a string |
| replace() | Replaces the first matching occurrence |
| split() | Splits a string into an array |
| charAt() | Returns a character at a specified index |

**Example:**

```
None
let name = "  Divya  ";

console.log(name.trim());
console.log(name.toUpperCase());
console.log(name.toLowerCase());
```

# 18. DOM – Document Object Model

DOM stands for Document Object Model.

It represents an HTML document as a tree of objects. JavaScript uses the DOM to access, modify, add, or remove HTML elements and their content.

## 18.1 Selecting HTML Elements

**getElementById():** Selects an element using its ID.

```
None
document.getElementById("heading");
```

**getElementsByClassName():** Selects elements using their class name.

```
None
document.getElementsByClassName("box");
```

**getElementsByTagName():** Selects elements using their tag name.

```
None
document.getElementsByTagName("p");
```

**querySelector():** Selects the first element matching a CSS selector.

```
None
document.querySelector(".box");
```

**querySelectorAll():** Selects all elements matching a CSS selector.

```
None
document.querySelectorAll(".box");
```

### 18.2 Modifying HTML Content

**innerHTML:** Gets or sets HTML content inside an element.

```
None
document.getElementById("heading").innerHTML = "Welcome";
```

**textContent:** Gets or sets the text content of an element.

```
None
document.getElementById("heading").textContent = "Hello World";
```

### 18.3 Modifying CSS Styles

JavaScript can change the CSS styles of HTML elements.

```
None
let box = document.querySelector(".box");
```

```
box.style.backgroundColor = "blue";
box.style.color = "white";
box.style.padding = "20px";
```

### 18.4 Adding and Removing Classes

The `classList` property is used to manage CSS classes of an HTML element.

**add():** Adds a class.

```
None
box.classList.add("active");
```

**remove():** Removes a class.

```
None
box.classList.remove("active");
```

**toggle():** Adds a class if it does not exist or removes it if it already exists.

```
None
box.classList.toggle("active");
```

**contains():** Checks whether an element contains a particular class.

```
None
console.log(box.classList.contains("active"));
```

## 19. JavaScript Events

An event is an action or occurrence detected by the browser, such as a click, key press, or form submission.

### 19.1 Common Events

| Event | Description |
| --- | --- |

| | |
|---|---|
| onclick | Occurs when an element is clicked |
| onchange | Occurs when an input value changes and is committed |
| oninput | Occurs whenever the input value changes |
| onsubmit | Occurs when a form is submitted |
| onmouseover | Occurs when the mouse moves over an element |
| onmouseout | Occurs when the mouse moves out of an element |
| onkeydown | Occurs when a key is pressed |
| onkeyup | Occurs when a key is released |
| onload | Occurs when a resource or page finishes loading |

## 19.2 Handling Events Using addEventListener()

`addEventListener()` attaches an event handler to an HTML element.

```
None
<button id="btn">Click Me</button>

<script>
    let button = document.getElementById("btn");

    button.addEventListener("click", function() {
        alert("Button Clicked");
    });
</script>
```

## 19.3 Event Object

The event object contains information about an event.

```
None
document.getElementById("btn").addEventListener("click",
function(event) {
    console.log(event.target);
});
```

Here, `event.target` identifies the element that triggered the event.

# 20. Form Handling and Validation

JavaScript is commonly used to validate user input before sending form data to a server.

**Example:**

```
None
<form id="studentForm">
    <input type="text" id="name" placeholder="Enter Name">
    <button type="submit">Submit</button>
</form>

<script>

document.getElementById("studentForm").addEventListener("submit",
function(event) {
        event.preventDefault();

        let name = document.getElementById("name").value;

        if (name.trim() === "") {
            alert("Name is required");
        } else {
            alert("Form Submitted Successfully");
        }
    });
</script>
```

**Important concepts:**

- `value`: Gets the current value of an input element.
- `preventDefault()`: Prevents the browser's default action.
- `trim()`: Removes whitespace from the beginning and end of a string.
- `submit`: Event triggered when a form is submitted.

# 21. Error Handling

Error handling is used to manage runtime errors and prevent unexpected program termination.

## 21.1 try Block

Contains code that may produce an error.

### 21.2 catch Block

Handles errors that occur in the try block.

### 21.3 finally Block

Executes whether an error occurs or not.

**Example:**

```
None
try {
    let result = 10 / 0;

    console.log(result);
} catch (error) {
    console.log("An error occurred:", error.message);
} finally {
    console.log("Execution completed");
}
```

### 21.4 throw Statement

Used to generate a custom error.

```
None
let age = 15;

if (age < 18) {
    throw new Error("Age must be 18 or above");
}
```

## 22. JSON – JavaScript Object Notation

JSON stands for JavaScript Object Notation.

It is a lightweight data format used to exchange data between clients and servers.

**Example JSON:**

```
None
{
    "id": 101,
    "name": "Divya",
    "department": "CSE"
}
```

### 22.1 JSON.stringify()

Converts a JavaScript object into a JSON string.

```
None
let student = {
    name: "Divya",
    age: 21
};

let jsonData = JSON.stringify(student);

console.log(jsonData);
```

### 22.2 JSON.parse()

Converts a JSON string into a JavaScript object.

```
None
let jsonData = '{"name":"Divya","age":21}';

let student = JSON.parse(jsonData);

console.log(student.name);
```

## 23. Asynchronous JavaScript

Asynchronous JavaScript allows operations to execute without blocking the main thread while waiting for operations such as API requests or timers.

### 23.1 setTimeout()

Executes a function once after a specified delay.

```
None
setTimeout(function() {
    console.log("Hello");
}, 2000);
```

### 23.2 setInterval()

Executes a function repeatedly at a specified interval.

```
None
setInterval(function() {
    console.log("Running");
}, 2000);
```

### 23.3 Callback Function

A callback is a function passed as an argument to another function and executed later.

```
None
function greet(name, callback) {
    console.log("Hello " + name);
    callback();
}

function message() {
    console.log("Welcome");
}

greet("Divya", message);
```

### 23.4 Promise

A Promise represents the eventual completion or failure of an asynchronous operation.

A Promise has three states:

- Pending – Operation is still in progress.
- Fulfilled – Operation completed successfully.
- Rejected – Operation failed.

**Example:**

```
None
let promise = new Promise((resolve, reject) => {
    let success = true;

    if (success) {
        resolve("Operation Successful");
    } else {
        reject("Operation Failed");
    }
});

promise
    .then(result => console.log(result))
    .catch(error => console.log(error));
```

### 23.5 async and await

`async` and `await` provide a cleaner way to work with Promises.

```
None
async function getData() {
    let response = await
fetch("https://jsonplaceholder.typicode.com/users");

    let data = await response.json();

    console.log(data);
}

getData();
```

- `async`: Declares an asynchronous function that returns a Promise.
- `await`: Waits for a Promise to settle and returns its fulfilled value or throws its rejection reason.

## 24. Fetch API

The Fetch API is used to send HTTP requests and receive responses from servers.

It is commonly used to communicate with ASP.NET Core Web APIs.

### 24.1 GET Request

Used to retrieve data from a server.

```
None
fetch("https://jsonplaceholder.typicode.com/users")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));
```

### 24.2 POST Request

Used to send new data to a server.

```
None
fetch("https://localhost:7057/api/movies", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify({
        title: "Leo",
        genre: "Action"
    })
})
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.log(error));
```

**Important Fetch concepts:**

- `fetch()`: Sends an HTTP request.
- `method`: Specifies the HTTP method.
- `headers`: Specifies request metadata.
- `body`: Contains the data sent to the server.
- `JSON.stringify()`: Converts JavaScript data into JSON.
- `response.json()`: Reads the response body as JSON.

Note: Fetch does not automatically reject a Promise for HTTP error status codes such as 400 or 500. Check `response.ok` to handle these responses.

# 25. Local Storage and Session Storage

Web Storage allows browsers to store data on the client side.

## 25.1 Local Storage

Stores data without an automatic expiration time. Data remains available across browser sessions until it is cleared.

```
None
localStorage.setItem("username", "Divya");

let name = localStorage.getItem("username");

localStorage.removeItem("username");

localStorage.clear();
```

## 25.2 Session Storage

Stores data for the current browser tab session. It is generally cleared when that tab or window session ends.

```
None
sessionStorage.setItem("username", "Divya");

let name = sessionStorage.getItem("username");

sessionStorage.removeItem("username");

sessionStorage.clear();
```

## 25.3 Difference Between Local Storage and Session Storage

| Feature | Local Storage | Session Storage |
| --- | --- | --- |
| Lifetime | Until cleared | Current tab session |
| Browser restart | Data remains | Session data normally ends |
| Storage location | Browser | Browser |
| Data sharing | Same origin across tabs | Separate per tab |

| | | |
|---|---|---|
| Storage type | Key-value pairs | Key-value pairs |

# 26. JavaScript Modules

Modules allow JavaScript code to be divided into separate files and reused through imports and exports.

## 26.1 Export

Used to make variables and functions available to other files.

**math.js**

```
None
export function add(a, b) {
    return a + b;
}
```

## 26.2 Import

Used to access exported members from another file.

**app.js**

```
None
import { add } from "./math.js";

console.log(add(10, 20));
```

**Advantages:**

- Code reusability.
- Better organization.
- Easier maintenance.
- Separation of responsibilities.